杂项板子

生成时间: 2026-09-05

默认省略通用 C++ 头文件与空壳 main

目录

1 动态规划 3
1.1 动态规划/对 dp 的一些思考 3
1.2 动态规划/dp 笔记/数位 dp 笔记 3
1.3 动态规划/dp 笔记/普通 dp 常见状态 3
1.4 动态规划/dp 笔记/状压 dp 常见状态 4
2 数据结构 5
2.1 数据结构/数据结构笔记/数据结构 trick 5
3 图论 6
3.1 图论/Trick/图相关 trick 6
3.2 图论/图论笔记/图论笔记(几类特殊图) 6
4 博弈论 11
4.1 博弈论/nim 游戏 SG 函数 11
5 通用 Trick 与杂项 14
5.1 其他/Trick/杂项相关 trick 14
5.2 计算几何(浮点数版)/Trick/计算几何 trick 15
6 字符串 17
6.1 字符串/笔记/一些比较神秘的 hash 手法 17
6.2 字符串/笔记/字符串 trick 18

1 动态规划

1.1 动态规划/对 dp 的一些思考

有时候想想 这个状态是否完备？ 状态数是不是可以通过某些手段缩减?

房子 dp 2 原则 1.缺啥状态补啥状态 2.考虑如何刻画/转化这个操作/答案

多位 dp 的时候能把小的项放在前面 会快一点(?)

状压子集枚举 3^n st 的子集 for(int i=st;i;i=(i-1)&st) st 的超集 for(int i=st;i<(1<<n);i=(i+1)|st)

退背包 一般用于我要统计不包含一个物品的方案数 就是反着做背包就行 ->

```
    auto add=[&](int cnt,int siz,int val)->void{
        for(int i=cnt;i>=1;i--){
            for(int j=siz;j>=val;j--){
                dp[i][j]+=dp[i-1][j-val];
            }
        }
    };
    auto del=[&](int cnt,int siz,int val)->void{
        for(int i=1;i<=cnt;i++){
            for(int j=val;j<=siz;j++){
                dp[i][j]-=dp[i-1][j-val];
            }
        }
    };
```

前后缀背包?

博弈论 dp 我们可以仿照 sg 函数的感觉进行状态设计 dp[i]=1 表示先手必胜 dp[i]=0 表示先手必败 重要是先后手之间的传递

概率/期望 dp 最重要的是写出转移式子,如果出现了自指的情况，可以移项

状压 dp 能做这样的事情 1.枚举阶乘的顺序 2.拓扑序计数

1.2 动态规划/dp 笔记/数位 dp 笔记

1.2.1 数位 DP 常用形参

以下为记忆化搜索函数 `dfs` 的常设定的形参。

- pos：`int` 型变量，表示当前枚举的位置，一般从高到低。
- limit：`bool` 型变量，表示枚举的第 `pos` 位是否受到限制。
  - 为 `true` 表示取的数不能大于 `a[pos]`，而只有在 `[pos+1, len]` 的位置上填写的数都等于 `a[]` 时该位才为 `true`。
  - 否则表示当前位没有限制，可以取到 `[0, R-1]`，因为 `R` 进制的数中数位最多能取到的就是 `R-1`。
- last：`int` 型变量，表示上一位（第 `pos+1` 位）填写的值。
  - 往往用于约束相邻数位之间的关系的问题。
- lead0：`bool` 型变量，表示是否有前导零，即在 `len \to (pos+1)` 这些位置是不是都是前导零。
  - 基于常识，我们往往默认一个数没有前导零，也就是最高位不能为 0，即不会写为 `000123`，而是写为 `123`。
  - 只有没有前导零的时候，才能计算贡献。
  - 那么前导零可用时答案有关？
    - 统计 `0` 出现次数。
    - 相邻数位的差值。
    - 以后问题为起点确定的奇偶位。
- sum：`int` 型变量，表示当前 `len \to (pos+1)` 的数位和。
- r：`int` 型变量，表示整数 `x` 取模某个数 `m` 的余数。
  - 该参数一般会固定在：约束中出现了“能被 m 整除”。
  - 当然也可以拓展成取数权取模的结果。
- st：`int` 型变量，用于状态压缩。
  - 对一个集合的数位权上的出现次数的奇偶性有要求时，其二进制形式就可以表示每个数出现的奇偶性。

1.3 动态规划/dp 笔记/普通 dp 常见状态

有些 dp 状态可以奇偶项不同（

dp[len][j][k] 表示长度 len 满足尾巴是 j,k j<k 的序列数 类似的记录尾部状态的 dp 有很多

dp[i][j] 表示将整数 j 划分为 i 个不同的正整数的方案数 <=> 等效命题：存在划分 l1,l2,l3,l4..lk 满足 l1+l2+l3+l4+…+lk=j,l1<l2<l3<…两者是双射 转移 考虑一个 j 的划分{l1,l2,l3…li} case1> 划分中不存在 1 dp[i][j] <- dp[i][j-i] 整体-1 case2> 划分中存在 1 dp[i][j] <- dp[i-1][j-i] 整体-1,且划分中 1 的个数-1

dp[i][k] 表示合法的数量为 i 的前缀某排列子集末尾的 rk 为这个前缀排列中 rk=k 的方案数 –>解决某类值域上的相邻约束排列计数问题 一类计数问题，我们可以不考虑放具体的数，而是考虑相对位置，这和具体数是一个双射关系。 https://atcoder.jp/contests/abc457/tasks/abc457_f

树上背包经典状态 dp[u][i] 表示以 u 为根的子树中，背包大小为 i 的… 复杂度是经典的 on 方 为啥??? 因为背包合并本质笛卡尔积每一对节点 (x,y) 只在 lca(x, y) 处被计算一次，所以复杂度是 $O(n^2)$ 的

一类异或=0 的 dp dp[i] 表示元素异或=0 的方案数 可以钦定 i=x 利用容斥来转移 具体来说 考虑 dp[i-2] dp[i-1]的含义 然后再考虑组合意义

一类路径选址的 dp(k 聚类,IOI 1996 邮局问题) 通常我们先考虑一个区间的代价，再按划分 dp/连续段 dp 的套路去做

1.4 动态规划/dp 笔记/状压 dp 常见状态

dp[st][i] 表示经过图中某些点构成的集合为 st,且最后一个点是 i 的某性质

2 数据结构

2.1 数据结构/数据结构笔记/数据结构 trick

线段树可以维护的带结合律的操作 摩尔投票，动态树直径,区间等差数列加,区间最大子段和，ddp 摩尔投票维护绝对众数（区间内出现次数严格大于一半的数）：维护每个节点的候选众数和净剩票数，合并时攻击抵消或合并，注意验证是否为绝对众数，可以维护下标然后二分。动态树直径：考虑点作为原树的点，区间作为覆盖点的直径，考虑两个直径的合并(u,v),(x,y),新的直径只可能是(u,v),(x,y),(u,x),(v,y),(u,y),(v,x)这六种情况，合并时枚举这六种情况取最大值即可。区间等差数列加：节点维护 sum,首项，公差，lazytag 维护首项和公差，tag 合并直接加，而下放直接修正偏移量就行 如果没有单点修/单点查，可以用二阶差分，考虑 d1[i]=a[i]-a[i-1],d2[i]=d1[i]-d1[i-1],首项 k，公差 d 修改是：d2[l]+=k,d2[l+1]+=d-k,d2[r+1]-=k+(r-l+1)d,d2[r+2]+=k+(r-l)d 区间最大子段和:维护四个信息:区间的最大后缀和，区间的最大前缀和，区间和，区间最大子段和 区间带修合法括号匹配：直接维护前缀和和区间 min 就行

查询一个区间是否包含一个区间，这个事情是可以二维数点做的，具体来说，把询问离线， 按 r 排序，用 bit 维护 l 的位置就行，在线可以用主席树维护。

SWAG 用来解决这样的问题 维护一个满足结合律的元素的窗口。即 窗口内是结合的元素 例如 on 来做这样一个问题 固定 k 的窗口查询所有窗口的 max 的 xor 或者 查询最长的满足某个矩阵乘法有某类特征的子序列

滑动窗口维护定长区间最大值：单调队列

区间染色：给定一个长度为 n 的数组，初始为空白。有 m 次操作，每次将区间 $[l,r]$ 染成某种颜色 c。后续的染色会覆盖之前的染色，最后求所有位置的最终颜色。 这个事情是可以 DSU 做的，具体来说，把询问离线，倒序处理这个东西，fa[i]表示从位置 i 开始（包含 i），右侧第一个未被染色的位置，处理询问就暴力做就行，即我们先找到 j=find(l),如果 j<=l,fa[j]=find(j+1),然后直接做就行。

查询 子集中 xor 和最大的子集 可以线性基 具体来说 直接从大到小合并线性基即可

给定一个 n 的序列 值域是小的 我们 O(sqrt(n))单点修 O(1)区间查值为 x 的数的个数 具体来说 分块 维护块内 x 的前缀和 再维护块之间的前缀和 查询是简单的 整块前缀和+散块前缀和 修改是简单的 先修块内的 再修块之间的

nxti 是很好用的东西 经常在扫描线/线段树类似题中见到。

3 图论

3.1 图论/Trick/图相关 trick

树上任意一点 v，距离其最远的点一定是该树某条直径的两个端点之一。 树上任意一点到一个点集的最远距离，必然是到该点集直径的两个端点之一的距离。 考虑反证法即可

联通块 https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79644138&returnHomeType=1&uid=719203876

有时候想想菊花图呢

每个点只有一个出边是内向基环树，可以倍增 每个点只有一个入边是外向基环树

最小瓶颈路：一个点到一个点所有路径中最大点权的最小值，可以用 dijk 算 多次询问的话可以求出 mst 然后跑树剖/rmq

以 u 为根的子树拓扑序列的计数是(n-1)!/所有子树大小的乘积(除 u)

一个有向树的存在 i->j 边，则他的传递闭包中(到达 j 的集合)交(i 可达的集合).size()=2; 如果按传递闭包的 sz 排序，这个是天然符合拓扑序的。

一个无向树的传递闭包，按行按集合划分的等价类是描述同链的

欧拉图：存在一条欧拉回路，能经过每个边一次，且起终点都是同一个点 半欧拉图：存在一条欧拉路径，能经过每个边一次

无向图是欧拉图的充要：所有点的度数都是偶数 无向图是半欧拉图的充要：恰好有 2 个顶点的度数是奇数，其余所有顶点的度数都是偶数

有向图是欧拉图的充要：所有点的出度等于入度 有向图是半欧拉图的充要：一个点的出度-入度=1（起点），另一个点的入度-出度=1（终点），其余所有点的出度等于入度

注意上述的连通条件只要求所有有边相连的点在同一个连通块内，也就是忽略孤立点后基图连通；有向图判断连通时忽略边的方向。

01bfs:deque 012bfs:对每个边权开桶即可

同余最短路：其实就是拓展状态的 dijk，dis[u][k]表示到 u 点，模 2*w 的余数是 k 的最短路 w 是与 1 相连的最小边权，通常处理可以重复走边的可行性问题 当然也可以解 形如 ax+by+cz…<=h 的解个数问题 ，具体来说 不妨令 x 为最小的 建 i->i+y%x w=y 的边 跑 dijk 即可。

对于一颗树上如果有 k 个点，一定能让其两两配对，形成 floor(k/2)条边不交的路径，具体证明是 dfs 上传标记，在 u 处直接配对，这样每颗子树最多剩一个 推广结论：一个点双中从一点出发必定能构成 floor(deg/2)条边不交简单环

一个点双去掉某个点后还是联通的。点双本质是边的划分。

对于一个图来说，sigma_(u,v){min(deg[u],deg[v])}<=msqrt(m); 推广结论：一个图的三元环计数<=msqrt(m)，按度数小到大(相同按编号)定向后每个点的出度<=sqrt(m)

无向图的非树边只是返祖边，于是找环的时候可以遇到返祖边(u,v)的时候，直接从 u 跳 fa 到 v 即可 无向图的任意一条 s 到 t 路径的异或和=s 到 t 的简单路径异或和 xor 一些基本环的异或和 这边的基本环指的是某条返祖边在他的 dfs 生成树上构成的环

最大权闭合子图：闭合子图是这样的东西，在一个有向图中，如果选取了一个点集 V'，且对于 V' 中的任意一个点 u，其所有的出边 (u,v) 指向的终点 v 也都必须在 V' 中，那么这个点集及其内部的边就构成了一个闭合子图。 最大权闭合子图：图中每个节点都有一个权值（可正、可负、可为零）。在所有的闭合子图中，节点权值之和最大的那个子图，就是最大权闭合子图。 建图方法：建立超级源点 S 和超级汇点 T。对于原图中的每个节点 i：如果权值 $w_i > 0$：连一条从 S 到 i 的边，容量为 w_i。如果权值 $w_i < 0$：连一条从 i 到 T 的边，容量为 $-w_i$。 如果权值 $w_i = 0$：可忽略（或视作容量为 0 的边）。 对于原图中的每一条有向边 (u,v)：连一条从 u 到 v 的边，容量为 ∞（无穷大）。 求解结论：

$$最大权值和 = \sum(所有正权点的权值) - 建出网络的最小割（即最大流）$$

这样其实保证了，容量无穷的边不会被割。 最小割：分离 S 集和 T 集最小的边容量和。

在任何森林（包含树）中，对于一个点集 S，其在图中的连通块数量等于 点数减去在集合内部的边数。

其实我们一个双关键字 也可以作为 dijk 的距离 只要有偏序关系就行

3.2 图论/图论笔记/图论笔记(几类特殊图)

3.2.1 记几类特殊图

被傻逼 abc e 题击败了 遂记录之

3.2.1.1 0. 一些概念

1). 团：一个完全子图

2). 最大独立集：一个点数最多的顶点集，其中任意两个顶点之间都没有边

可以发现这两是对偶关系

3). 最小染色：用最少的颜色给点染色使得所有边连接的两点颜色不同。

4). 色数：最小染色中使用的颜色数

其实下面所述都属于完美图，即最大团=最小染色数

3.2.1.2 1. 可比图 -> 逆序图

可比图：刻画偏序关系中“是否可比”的无向图。它不是 Hasse 图：Hasse 图只保留覆盖关系边，而可比图等价于先取偏序关系的传递闭包，再忽略方向。

两元素之间可比，则存在一条无向边。

通常我们会画成有向图（如果定义了偏序关系），但本质是无向的（只关心可比关系）

链：构成一条链的元素两两之间可比 ⇒ 对应图中的团。

反链：构成一条反链的元素两两之间不可比 ⇒ 对应图中的独立集。

链划分：将一个偏序集划分成若干条链。

Dilworth 定理：最小链覆盖数 = 最大反链的大小 = 最大独立集的大小

对偶的 Dilworth 定理：最小反链覆盖数 = 最长链的大小 = 最大团的大小

其实此处的最长链是好求的，就是 DAG 的最长路

最大反链也是好求的，跑个传递闭包，按二分图的方法跑最小链覆盖就行

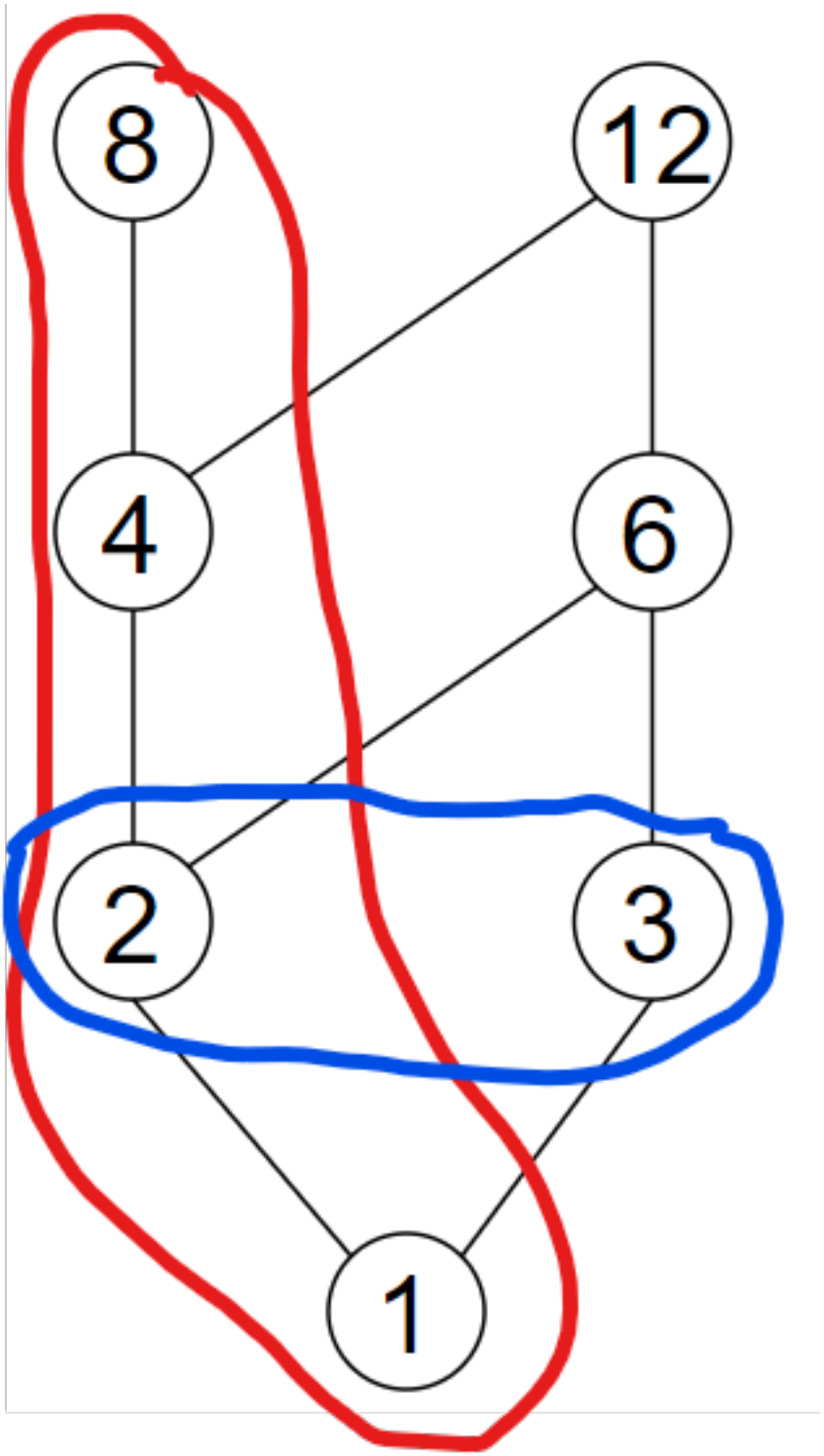


图 1　alt text

如果图中画的是 Hasse 图，要把传递可达的点对也视为可比；可比图会补上所有传递边。

这是个整除图来着，红色的是链 蓝色的是最大反链 注意到最小链划分数是 2，最大反链的元素数目也是 2

逆序图：给定一个序列，逆序图是表示序列中元素之间逆序关系的图

具体来说，对于序列中的元素 `a[i]` 和 `a[j]`，如果 `i<j` 且 `a[i]>a[j]`，则存在一条从 i 到 j 的无向边。

我们把逆序图套到可比图上，发现，逆序图的最长反链（最大独立集）就是最长上升子序列。

因为最长上升子序列中任意两个元素之间都不存在逆序关系，即两两不可比。

同理我们把这个序列划分成最少的下降子序列，会发现，这个个数正好是最长上升子序列的长度。

即 Dilworth 定理：最小链覆盖数 = 最大反链的大小

同理，逆序图的最长链（最大团）就是最长下降子序列，其中任意两个元素之间都存在逆序关系，即两两可比。

同理我们把这个序列划分成最少的上升子序列，会发现，这个个数正好是最长下降子序列的长度。

即 对偶的 Dilworth 定理：最小反链覆盖数 = 最长链的大小

所以我们看到可比图的时候，先想想这个可比图的链是啥含义，反链又是啥含义，这样有可能把一般可比图的求最大反链的 $O(n^{1/2}m)$ 优化成 $O(n\log n)$

3.2.1.3 2. 二分图

详细证明见二分图最大匹配.cpp 注释部分

二分图最大匹配：最大流 $O(n^{1/2}m)$

二分图最小点覆盖：在一张无向图中选择最少的顶点，满足每条边至少有一个端点被选。

二分图中，最小点覆盖中的顶点数量等于最大匹配中的边数量。（Kőnig 定理）

二分图最大独立集：在一张无向图中选择最多的顶点，满足两两之间互不相邻。

二分图中，最大独立集中的顶点数量等于 n - 最小点覆盖中的顶点数量。

其实对于一般图来说，最大独立集也 = n - 最小点覆盖中的顶点数量

有向无环图最小路径覆盖：在一张有向图中，选择最少数量的简单路径，使得所有顶点都恰好出现在一条路径中。

有向无环图的最小路径覆盖数等于顶点数减去最大匹配数。

通过 DAG 构造的二分图如下：

将每个顶点拆成两个顶点，`v_in`、`v_out`。

对于原图中的每条有向边 $u \to v$，在二分图中连边 `v_in-u_out`。

也就是说 对于一般的 DAG 我们可以通过这个转换 最大流 $O(n^{1/2}m)$ 求出最小路径覆盖，路径只能沿原图边走。

偏序集的最小链划分不是普通 DAG 原边路径覆盖，链内只要求前后可比；如果图里只给了 Hasse 边或普通 DAG 边，需要先跑传递闭包，再按可达关系建二分图。

3.2.1.4 3. 区间图 / 一堆区间的简单问题

区间图：给定 n 个区间，区间图是表示区间之间相交关系的图。

区间有交集则连边。

注意到区间图没有长度 ≥ 4 的无弦环，所以他是弦图

随便画画图就能证明

1). 最大团：直线上重叠层数最多的位置 <-> 最小染色：给每个区间染色使得重叠区间颜色不同，最少颜色数

此处的最大团=最小染色比较显然

经典模型：求区间图最大团

把区间变成 `{l,+1},{r,-1}`，然后对坐标排序，注意相同的时候 $+1$ 放前面即可，做扫描线即可

经典模型：求区间最小染色的方案

建立小根堆，里面存 `{某颜色的结束时间，颜色 ID}`，每次取出堆顶，如果堆顶的结束时间小于当前区间的开始时间，则该区间可以染该颜色，否则该区间无法染色，需要新开一个颜色，然后加入堆中

2). 最大独立集：求最多的不交区间数

这是经典的，按右端点 R 从小到大排序，选了 i 后，下一个选 $L_j > R_i$ 的第一个 j。

当然当我们要求最小点覆盖的时候只要用 n 减去最大独立集即可。

3). 选最少的点，使得每个区间内至少包含一个点。

经典贪心：按右端点 R 从小到大排序。如果当前区间不交，贪心地选该区间的最右端点（因为它能尽可能多地向右覆盖后续区间的左端点）。

当然其实我们知道 2 和 3 是等价的，你可以发现他们的本质是一样的。

4). 别的附赠小甜品

给定大区间 $[S,T]$ 和 N 个小区间，求最少选几个小区间能完全覆盖 $[S,T]$

按 l 排序，维护当前已覆盖到的右边界 r，在当前区间中找最大的 r 即可

每个区间 $[L_i,R_i]$ 有权值 W_i，求互不相交的区间集合使得权值和最大

随便 dp 即可

3.2.1.5 4. 弦图
弦图：没有大于 3 的“无弦环”的图，即他没有大洞

树也是弦图!

1). 最大势算法（MCS）生成完美消除序列（PEO）

完美消除序列：

序列 $v_1,...,v_n$ 中，对于任意 v_i，其在序列后方（$v_{i+1}...v_n$）的所有邻居构成一个团（即两两相连）。

给图中的每个点分配一个编号，n 倒数到 1 进行编号。

贪心策略：每次从未编号的节点中，选择与“已编号节点”相连边数最多的那个点进行编号。

感性理解一下，每次选相连边数最多的那个点可以保证团的纯度

我们需要维护一个 $label[i]$：表示节点 i 目前与多少个已编号的节点相连。

朴素实现是 $O(n*n)$ 的，不够优秀，我们可以用一个懒更新的手法把他变成 $O(n+m)$。具体来说，我们用一个桶 $st[i]$ 存储 $label=i$ 的点，然后维护一个 $mxlab$ 指向桶的末尾，再开个 $vis[i]$ 表示 i 是否被编号过，这样我们每次在取的时候可以检查 vis 数组，不成立的废弃即可。

注意到，边 (u,v) 最多贡献一次 $label$ 值，st 的 push 次数不超过 m，$mxlab$ 的回退总量 $\leq n$（一个点的邻接点数 $\leq n-1$），所以复杂度是 $O(n+m)$ 的。

2). 验证完美生成序列

考虑优化一下 PEO 的定义，把他写成归纳的形式：

对于当前点 u，我们不需要检查其后方邻居是否“两两”有边。我们只需要找到 u 在序列后方位置最靠前的那个邻居，记为 p。

只要满足：u 的所有其他后方邻居，都是 p 的邻居，那么 u 就满足性质。

随便归纳一下就行，类似于 n 循环比赛

考虑实现。

用 rk 反映射 peo 序列，$rk[i]$ 表示 i 在 peo 中的位置，我们处理出每一个 p 的检查列表 $chklis[p]$，然后按 peo 序列的顺序（其实这个是不必要的，按 1-n 都可以），遍历 p，给 p 的邻居打上时间戳，然后在对检查列表里的点检查邻居即可。

注意到每个 u 只属于一个 p，所以时间复杂度是 $O(n+m)$ 的。

3). 求解最大团，色数和最大独立集

根据完美图性质，最大团=最小染色数。

这个最大团已经在生成 PEO 的过程中算出来了。

回忆 $label[i]$：表示节点 i 目前与多少个已编号的节点相连。

根据弦图性质，这 $label[u]$ 个邻居加上 u 自己，恰好构成了一个大小为 $label[u]+1$ 的团。

最大独立集，贪心着求，开 vis 数组，按 PEO 选尽量前面即可。

感性理解，考虑交换论证，设 u,v，u 的出现早于 v，u 的团包含 v，考虑选 u，团内只能选一个，不劣，考虑选 v，扼杀了 v 构成的团内的某些可能，劣。

4 博弈论

4.1 博弈论/nim 游戏 SG 函数

4.1.1 尼姆（Nim）游戏

Nim 游戏的规则是这样的：地上有 n 堆石子，甲、乙两人交替取石子。每人每次只能从任意一堆石子里面取，至少取 1 枚，不能不取。最后没有子可取的人就输了。假如甲是先手，且已知每堆石子的数量 a_i，问是否存在先手必胜的策略。

4.1.1.1 结论

- 若初态为必胜态（$a_1 \oplus a_2 \oplus \cdots \oplus a_n \neq 0$），则先手必胜；
- 若初态为必败态（$a_1 \oplus a_2 \oplus \cdots \oplus a_n = 0$），则先手必败。

4.1.1.2 定理证明

定理 1：必胜态的后继状态至少存在一个必败态

证明：

设 $a_1 \oplus \cdots \oplus a_n = s \neq 0$，设 s 的二进制位为 1 的最高位是第 k 位，则 $a_1, \cdots, a_n$ 中一定有奇数个 a_i 的二进制的第 k 位为 1。

用 $a_i \oplus s$ 去替换 a_i，则：

$$\begin{aligned} a_1 \oplus \cdots \oplus (a_i \oplus s) \oplus \cdots \oplus a_n &= a_1 \oplus \cdots \oplus a_i \oplus s \oplus \cdots \oplus a_n \\ &= s \oplus s \\ &= 0 \end{aligned}$$

这是一种必败态。

同时，因为 $a_i \oplus s < a_i$，是合法的替换。

定理 2：必败态的后继状态均为必胜态

证明：

设 $a_1 \oplus \cdots \oplus a_n = 0$，则相同位上 1 的个数为偶数。此时，无论减少哪个数，都会使得异或和 $\neq 0$。

必胜态与必败态交替出现，终态 $(0, 0, ..., 0)$ 是必败态。

递推边界：

- 所有堆都为 0 时，异或和为 0，必败态。
- 若当前局面的异或和不为 0，则一定能操作到异或和为 0 的局面，是必胜态。
- 若当前局面的异或和为 0，则任意一次合法操作后异或和都会变为非 0，是必败态。

4.1.2 台阶型 Nim 游戏

有 $1 \sim n$ 级台阶，第 i 级台阶上摆放 a_i 个石子，每次操作可将第 k 级台阶上的石子移一些到第 $k-1$ 级台阶上，移到第 0 级台阶（地面）的石子不能再移动。

如果一个人没有石子可以移动，他就输了，问先手是否必胜。

4.1.2.1 结论

若奇数台阶上的石子数异或和不为 0，即 $a_1 \oplus a_3 \oplus a_5 \oplus \cdots \neq 0$，则先手必胜；否则先手必败。

4.1.2.2 定理

定理 1：必胜态的后继状态至少存在一个必败态

定理 2：必败态的后继状态均为必胜态

4.1.2.3 证明

情况 1： 若初态奇数台阶石子异或和不为 0（必胜态），甲先手：

1. 甲选择某个奇数台阶，将其部分石子移到下一级偶数台阶，使得所有奇数台阶石子数的异或和为 0
2. 如果乙将偶数台阶的石子移到奇数台阶，甲只需将乙移动的同等数量石子继续下移到下一级偶数台阶，保持奇数台阶异或和为 0
3. 如果乙将奇数台阶的石子移到偶数台阶，甲可以调整其他奇数台阶的石子，使得奇数台阶异或和重新为 0
4. 重复上述过程，最终甲将把第 1 级台阶的石子全部移到地面，获得胜利

情况 2： 若初态奇数台阶石子异或和为 0（必败态），甲先手：

- 无论甲如何操作，都会破坏奇数台阶石子异或和为 0 的状态
- 乙总是可以恢复奇数台阶石子异或和为 0 的状态
- 最终乙将获得胜利

4.1.3 有向图游戏

给定一个有向无环图，图中只有一个起点，在起点上放一个棋子，两个玩家轮流沿着有向边推动棋子，每次走一步，不能走的玩家失败。

4.1.3.1 mex 运算 (minimum exclusion)

$$\operatorname{mex}(S)$$

为不属于集合 S 中的最小非负整数，

$$\operatorname{mex}(S) = \min\{x\} \quad (x \in N, x \notin S)$$

例如：

$$\text{mex}\left(\{0,1,2\}\right)=3$$

$$\text{mex}\left(\{1,2\}\right)=0$$

4.1.3.2 SG 函数
设状态（节点）x 有 k 个后继状态（子节点）$y_1,y_2,\cdots,y_k$，

$$SG(x)=\text{mex}\left(\{SG(y_1),SG(y_2),\cdots,SG(y_k)\}\right)$$

4.1.3.3 SG 定理
由 n 个有向图游戏组成的组合游戏，设起点分别为 $s_1,s_2,\cdots,s_n$，当

$$SG(s_1)\oplus SG(s_2)\oplus\cdots\oplus SG(s_n)\neq 0$$

时，先手必胜；反之，先手必败。

4.1.4 SG 定理证明
4.1.4.1 定理陈述
由 n 个有向图游戏组成的组合游戏，设起点分别为 $s_1,s_2,\cdots,s_n$，当

$$SG(s_1)\oplus SG(s_2)\oplus\cdots\oplus SG(s_n)\neq 0$$

时，先手必胜；反之，先手必败。
4.1.4.2 证明
证明思路类似于 Nim 游戏的证明，通过分析必胜态和必败态的转移关系。

定义：

- 令 $X=SG(s_1)\oplus SG(s_2)\oplus\cdots\oplus SG(s_n)$
- 终态：所有游戏都处于无法操作的状态，此时 $X=0$

1. 终态是必败态

当所有游戏都处于无法操作的状态时：

- 所有 $SG(s_i)=0$（根据 SG 函数定义）
- 因此 $X=0\oplus 0\oplus\cdots\oplus 0=0$
- 无法进行任何操作，当前玩家失败

2. 若 $\boldsymbol{X\neq 0}$，存在操作使 $\boldsymbol{X=0}$

设 $X=k\neq 0$，则存在某个 $SG(s_i)$ 使得 $SG(s_i)\oplus k<SG(s_i)$。

理由：

- 设 k 的最高位为第 m 位
- 存在 $SG(s_i)$ 的第 m 位为 1
- 则 $SG(s_i)\oplus k$ 的第 m 位变为 0，且更高位不变
- 因此 $SG(s_i)\oplus k<SG(s_i)$

操作：

- 选择该游戏 i
- 将其状态从 s_i 移动到某个后继状态 t，使得 $SG(t)=SG(s_i)\oplus k$
- 这样的 t 一定存在，因为根据 SG 函数定义，$SG(s_i)$ 是其后继状态 SG 值的 mex
- 这意味着 0 到 $SG(s_i)-1$ 的所有值都会出现在后继状态的 SG 值中

结果：

$$\begin{aligned}X'&=SG(s_1)\oplus\cdots\oplus SG(t)\oplus\cdots\oplus SG(s_n)\\&=SG(s_1)\oplus\cdots\oplus(SG(s_i)\oplus k)\oplus\cdots\oplus SG(s_n)\\&=(SG(s_1)\oplus\cdots\oplus SG(s_i)\oplus\cdots\oplus SG(s_n))\oplus k\\&=k\oplus k\\&=0\end{aligned}$$

3. 若 $\boldsymbol{X=0}$，任何操作都会使 $\boldsymbol{X\neq 0}$

假设当前 $X=0$，玩家选择游戏 i 并将其状态从 s_i 移动到 t。

分析：

- 如果 $SG(t)=SG(s_i)$，则异或和不变，仍为 0
- 但根据 SG 函数定义，$SG(s_i)$ 是其后继状态 SG 值的 mex
- 这意味着 $SG(s_i)$ 不会出现在后继状态的 SG 值中
- 因此 $SG(t)\neq SG(s_i)$

结果：

$$
\begin{aligned}
X' &= SG(s_1) \oplus \cdots \oplus SG(t) \oplus \cdots \oplus SG(s_n) \\
&= (SG(s_1) \oplus \cdots \oplus SG(s_i) \oplus \cdots \oplus SG(s_n)) \oplus SG(s_i) \oplus SG(t) \\
&= 0 \oplus SG(s_i) \oplus SG(t) \\
&= SG(s_i) \oplus SG(t) \neq 0
\end{aligned}
$$

4.1.4.3 结论

- 从 $X \neq 0$ 的状态，玩家总可以移动到 $X = 0$ 的状态
- 从 $X = 0$ 的状态，玩家只能移动到 $X \neq 0$ 的状态
- 终态 $X = 0$ 是必败态

因此，当且仅当 $X \neq 0$ 时，先手必胜。

证毕。

5 通用 Trick 与杂项

5.1 其他/Trick/杂项相关 trick

括号序列合法的充要是 +1 −1 trick-> 所有前缀和>=0 sum=0 –>能推导出 这样的串里一定存在() 子串 合法括号序列 可以转为括号树 具体来说 每个左括号节点++ 并且是目前栈顶的儿子 注意搞个虚拟源点 0

考虑 m 是一个奇数长度序列中位数的充要条件是 >=m 的数 > 小于 m 的数 && <=m 的数 > 大于 m 的数 于是我们可以考虑使用+1 −1 trick 然后算前缀和

更一般的 考虑中位数是下中位数 满足 >=m 的数 > 小于 m 的数 && <=m 的数 >= 大于 m 的数 考虑中位数是上中位数 满足 >=m 的数 >= 小于 m 的数 && <=m 的数 > 大于 m 的数

sqrt 对 longlong 精度不够 要用 sqrtl

某些区间问题 可以利用前后缀处理的思想 https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79677851&returnHomeType=1&uid=719203876

某些题可以转化成二进制来做 https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79675702&returnHomeType=1&uid=719203876 eg x <- floor(x/2) = x>>1

位运算计数某些经典技巧是拆位，每位独立贡献 https://ac.nowcoder.com/acm/contest/view-submission?submissionId=79091623&returnHomeType=1&uid=719203876

压位 ull st=((ull)a<<32)|b;

x-y<=x^y<=x+y

$$|S \oplus M| = |S \cup M| - |S \cap M|$$

S,M 是状压的集合

实际上 一个 x 异或上一个区间得到的数构成的连续段是 log 的 具体来说 考虑 log 个相同前缀的区间 e.g 11001=[00000,01111]+[10000,10111]+[11000,11001] 考虑他的一个后缀是任意的，所有这个连续段的上下界可以 O(1)算

对一个 x 异或一个区间，得到的上下界可以 log 算，参见 2026 校赛 ex 题

对一个 x 异或一个区间，得到的连续段是一个的充要条件好像很深刻，我看不懂现在。

如果你要算一个数对集合内数的 xor<=w,你可以对集合建 01tire 然后在 01tire 上跑 dfs

xorhash 通常存在于某些 2/1 性质 奇偶性质的题里面 括号序列?!?

有些题有奇偶不变量/其他的不变量

压维转化 0-base xm+y 1base (x-1)m+(y-1)+1

曼哈顿距离 dis(x1,y1,x2,y2)=abs(x1-x2)+abs(y1-y2) 切比雪夫距离 dis(x1,y1,x2,y2)=max(abs(x1-x2),abs(y1-y2)) 互转 常用 (x,y)->(u=x+y,v=x-y)，原坐标的曼哈顿距离等于新坐标的切比雪夫距离：abs(dx)+abs(dy)=max(abs(du),abs(dv)) 反变换 x=(u+v)/2,y=(u-v)/2，注意奇偶性；若直接映射到 ((x+y)/2,(x-y)/2)，则距离也会缩小一半

max(a,b)=(a+b+abs(a-b))/2 min(a,b)=(a+b-abs(a-b))/2

dilworth 定理 偏序集里，链指任意两个元素都可比的一组元素，反链指任意两个元素都不可比的一组元素。Dilworth 定理：最少用多少条链覆盖所有元素 = 最大反链大小。对偶形式：最少用多少个反链覆盖所有元素 = 最大链大小。若用 DAG 表示偏序，链内相邻元素只要求“可达/可比”，不一定要有原图直接边，所以必要时先做传递闭包再建二分图。通常考察二维偏序，按一维排序后，最大链/最大反链常可以转成 LIS/LDS；最少链划分则由最大反链给出。

hall 定理 设二分图左右点集为 X,Y。Hall 定理：存在一个匹配覆盖左侧全部 X 的充要条件是，对于任意 S 属于 X，都有 |N(S)|>=|S|。含义是：左侧任意一批点合起来能选择的右侧候选点，数量不能少于这批左侧点，否则鸽巢原理必然匹配不完。若 |X|=|Y|，并且存在覆盖 X 的匹配，则这个匹配大小为 |X|=|Y|，因此也是完美匹配；左右大小不等时只能说覆盖左侧，不叫完美匹配。在树上的推广：如果我们选择 k 个点和树的叶子做匹配，要满足 对 u 子树 子树中的选择的点<=子树的叶子数

排序不等式 simga ai*bi 最大的充要为 即对于所有 i, j，若 $a_i < a_j$ 则 $b_i < b_j$ 即 a 和 b 是同序的。 最小的充要为 即对于所有 i, j，若 $a_i < a_j$ 则 $b_i > b_j$ 即 a 和 b 是反序的。–同理 max(ai-bi) 最小 -> a 和 b 是同序的。https://www.luogu.com.cn/problem/AT_arc201_d

注意 %m 的形式有时候可以破环为链

排列诸结论 1.交换两个数的顺序，会使排列的逆序对数+1/-1 奇偶性变换 2.一个长度为 n 的排列，其逆序对数的奇偶性等于 n-c c 是置换环数 置换环：把排列看成映射 i->p_i，从一个下标不断跳到 p_i，直到回到起点得到的环；下标[l,r]包含了[l,r]的所有元素只是一个封闭块，不一定是单个置换环 3.交换大小为 x，y 的两个块 对逆序对的贡献为 x*y-2k,k 为原来块间的逆序对数 4.最优交换次数 K：将排列 P 分解为若干个不相交的置换环。设环的个数为 C，则将 P 排序为 $(1, 2, ..., N)$ 的最小交换次数为：

$$K = N - C$$

5.置换环内交换：如果你交换同一个环内的两个元素，这个环一定会分裂成两个更小的环。 6.置换环间交换：如果你交换不同环的两个元素，这两个环会合并成一个大环。

恰好的方案数难求的时候 考虑至少的方案数然后转化 经典的 求=k 的方案数=求>=k 的方案数-求>k 的方案数 经典的 k=simga_i<=k {1} 然后我们能把一个东西变成对每个>=k 都计算一遍 式子很像的时候考虑求出整体贡献然后对每个 i 取消贡献

有时候建出操作树可以降维打击!

二分图染色可以使用拓展域并查集 具体来说是拆点 x->x,x+n x 代表黑色 x+n 代表白色 (u,v) 连边的时候 merge(u,v+n) merge(u+n,v) 如果 same(u,v) 则说明两个点同色 不成立

float 有效位数是 6-7 位 double 有效位数是 15-16 位 long double 有效位数是 18-19 位

记得 sqrtl 和 atanl acos(-1)=pi

完全图最小生成树可以用 boruvka 算法 也可能可以用数据结构优化 prim 算法

树直径更新：考虑原树直径(u,v)，考虑树上新增的点，直径的更新只可能是(u,new),(v,new)或者(u,v)

x->floor(x/2),ceil(x/2) 在递归的每一层都是 v,v+1

经常忘的：根号做法，调和级数 根号做法 分块，定期重构，根号分治

安全取整 注意负数下取整是往负无穷，上取整是往 0 // floor(a/b) long long floor_div(long long a,long long b){ return a/b - ((a^b)<0 && a%b); }

\// ceil(a/b) long long ceil_div(long long a,long long b){ return a/b + ((a^b)>0 && a%b); } 正数 ceil a+b-1/b

一个经典转化是 一个升序列表 x1,x2,x3…xn y1=x1-1,y2=x2-2 注意到 yi 也是单增的。一个经典转化是 极长连续段数可以转化为有 01 的序列数，所以可以考虑某种形态的生成函数，例如 g(x,y)=x^v[i]*y^v[i+1]

位运算妙招：只遍历 st 中的所有 1，能把时间复杂度砍掉 2 的一个常数

```
for(int i=1;i<(1<<n);i++){
    int tp=i;
    while(tp){
        int j=__builtin_ctz(tp);
        if((st[j]&(i^(1<<j)))==st[j]) dp[i]+=dp[i^(1<<j)];
        tp&=(tp-1);
    }
}
```

矩形有交只需要长宽都有交即可 转化为线段交

adhoc 方法总括 1.寻找不变量 2.找最小操作

k 次区间翻转的本质是 选 k+1 个不相邻连续段，把他们合并成一个连续段

相邻长为 k 的区间有关系，我们可以想想按模 k 分治的子序列性质

考虑解耦变量呢?

$| x | = \max(x, -x)$

5.2 计算几何(浮点数版)/Trick/计算几何 trick

1. 多边形的面积 A

$$A = \sum_{i=1}^{N} A_i = \frac{1}{2} \sum_{i=1}^{N} (x_i y_{i+1} - x_{i+1} y_i)$$

>逆时针为正

2. 多边形重心

$$C_x = \frac{\sum_{i=1}^{N} (x_i y_{i+1} - x_{i+1} y_i)(x_i + x_{i+1})}{3 \sum_{i=1}^{N} (x_i y_{i+1} - x_{i+1} y_i)}$$

$$C_y = \frac{\sum_{i=1}^{N} (x_i y_{i+1} - x_{i+1} y_i)(y_i + y_{i+1})}{3 \sum_{i=1}^{N} (x_i y_{i+1} - x_{i+1} y_i)}$$

（注：式中当 $i = N$ 时，$x_{N+1} = x_1$，$y_{N+1} = y_1$）

3. 皮克定理：

$$S = I + \frac{B}{2} - 1$$

其中 S 是面积，I 是严格内部整点数，B 是边界整点数。

4. 一条向量为 $(\Delta x, \Delta y)$ 的边上的整点数为

$$\gcd(| \Delta x |, | \Delta y |)$$

>包含两个端点则+1，这里是为了方便去重这样写的。

5. 判凸包相交和计算凸包距离

   将凸包 A 和 B 的相交与距离问题转化为 $C = A - B$（即 A 和 $-B$ 的闵可夫斯基和）。判定相交：如果 A 和 B 相交，当且仅当原点 $(0,0)$ 在凸包 C 的内部或边界上。 计算距离：如果 A 和 B 不相交，它们之间的最短距离等于原点 $(0,0)$ 到凸包 C 边界的最短距离。

6 字符串

6.1 字符串/笔记/一些比较神秘的 hash 手法

6.1.1 一些比较神秘的 hash 手法

以下的 $Hash$ 均采用这个生成：

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count()^(ull)(new char));
vector<ull> h(n+1);
for(int i=1;i<=n;i++) h[i]=Xhash::splitmix64(rng())|((1ull<<50)+1);
```

XHash 类见 防 hack 的 umap,gp_hash_table.cpp

对字符集/数集中的每个元素都随机分配一个 $Hash$ 值（大奇数）。

采取自然溢出 $Hash$。

给定数组/字符串。

6.1.1.1 1.1. 考虑求解每个元素的出现次数为偶数的子数组/子串个数

老生常谈，维护前缀异或和就行，区间 $[l,r]$ 合法的条件是 $sum[r]\oplus sum[l-1]=0$，即 $sum[r]=sum[l-1]$。

如果他要你求区间能重排成回文串，做一些适当转化可以变成这个问题。

6.1.1.2 1.2. 更一般的，考虑求解每个元素的出现次数为 k 的倍数的子数组/子串个数

做一个前缀 $Hash$ 和，维护一个 cnt 数组，当数组中的某个元素达到 k 时候，在前缀和上减去 $(k-1)$*$h[i]$，这样每个前缀的意义其实是一个长度为 n 的状态向量，每个位置是模 k 的。区间 $[l,r]$ 合法的条件是 $sum[r]=sum[l-1]$。

实现：

```
for(int i=0;i<26;i++) w[i]=rng()|1;
for(int i=1;i<=n;i++){
    cnt[s[i]-'a']++;
    if(cnt[s[i]-'a']==k){
        ph[i]=ph[i-1]-w[s[i]-'a']*(k-1);
        cnt[s[i]-'a']=0;
    }
    else ph[i]=ph[i-1]+w[s[i]-'a'];
}
```

例题：https://codeforces.com/gym/105911/problem/E

6.1.1.3 1.3. 更特殊的，考虑求解每个元素的出现次数为 k 的子数组/子串个数

注意到我们维护一个滑动窗口，窗口里面的元素个数 $\le k$，然后问题就转化成 1.2 了。

例题：https://atcoder.jp/contests/abc455/tasks/abc455_g 子问题 1

6.1.1.4 2.1. 考虑求解字母个数相同的子数组/子串个数

对我们的 $Hash$ 做一个小小的修改，对前 $n-1$ 个元素保持不变，最后一个元素取前 $n-1$ 个元素哈希值和的相反数，做前缀 $Hash$，区间 $[l,r]$ 合法的条件是 $sum[r]=sum[l-1]$。

例题：https://atcoder.jp/contests/abc455/tasks/abc455_e

6.1.1.5 3.1 考虑判断一个子区间能够进行重排后变成另一个子区间

那么其实就是我们普通的前缀 $Hash$，判断 $[l1,r1]$ 和 $[l2,r2]$ 是否相等，即 $sum[r1]-sum[l1-1]=sum[r2]-sum[l2-1]$。

那其实说明我们前缀 hash 能代表某个多重集。

以上的 $Hash$ 概率都是没有问题的，防 $hack$ 也是对的。

6.1.1.6 4.1 考虑判断区间中的所有数的个数是否能整除其本身

直接取 $Hash(x)=1/x(modp)$，然后做前缀 $Hash$，多模几个大质数，判断在这些质数下区间和是否相等，并且是一个小值。

6.1.1.7 5.1 有根树同构(树 hash)

$$H(u)=1+\sum_{v\in son(u)} f(H(v)) \pmod{2^{64}}$$

其中 f 是一个非线性映射函数，比如 $splitmix64$。

使用例子：

```
Xhash64 H;
ull h[N];
void dfs(int u,int fa){
    h[u]=1;
    for(int v:g[u])
        if(v!=fa) dfs(v,u),h[u]+=H(h[v]);
}
```

6.1.1.8 5.2 无根树同构

对重心跑一边有根树同构，如果有两个重心，都跑一遍取 max 即可

6.1.1.9 5.3 有标号图同构

对每个标号随一个 $hash$ 值，然后每个标号的 $hash$ 值是他邻居的 $hash$ 值的和(自然溢出)，对每个标号检查即可。

6.2 字符串/笔记/字符串 trick

二分 hash 能做的 1.求 lcp 长度 2.比较字符串 3.找回文串的所有出现 考虑用 # 拼接串，然后枚举中心点，二分长度，hash 判断反串和正串即可

kmp 自动机 你可以把他当作一个包含单串所有前缀的 acam，其实也是一个单串 acam，只需要在他的所有前缀打上存在标记即可 他的转移边，指向的就是他的前缀接受了这个字符后转移到的位置 kmp 自动机 就是 nxt[j][c]，表示长度为 j 的前缀，接受字符 c 后转移到的前缀的长度 构建的原理跟 acam 是一致的

border：一个串的子串是他的真前缀和真后缀

对一个查询某个串位置的出现的问题，如果对于查询串的 len 的 sum 有限制的话，考虑 sqrt trick,因为对于某个长度串的出现次数是 on 的 而长度这里又有一个自然根号(即最劣构造 1,2,3,4…),所以出现次数是严格根号的，不过要注意去重。

字符串的本质不同字符串是 O(n)级别的。经典结论：新的本质不同回文串，必定伴随着当前已知的“最右回文边界”的扩展而产生。此处的本质不同指的是，该回文串在原串中第一次出现。具体来说 如果我们用马拉车处理出 n 个回文半径，再维护一个 maxR，新的回文串的右端点 r 如果大于 maxR，那么 r-maxR+1 到 r 的回文串就是新的本质不同回文串。但是其是一个必要条件，需要 hash 去重。